machines

MDPI

Article

Comparative Analysis of Object Detection Models for Edge Devices in UAV Swarms

Dimitrios Meimetis [1,2,*], Ioannis Daramouskas [2], Niki Patrinopoulou [2], Vaios Lappas [1] and Vassilis Kostopoulos [2]

[1] Department of Aerospace Science & Technology, National Kapodistrian University of Athens, 10563 Athens, Greece; valappas@aerospace.uoa.gr
[2] Applied Mechanics Lab, University of Patras, 26504 Patras, Greece; daramousk@ceid.upatras.gr (I.D.); n.patrinopoulou@ac.upatras.gr (N.P.); kostopoulos@upatras.gr (V.K.)
* Correspondence: dmeimetis@ceid.upatras.gr

Abstract

This study presented a comprehensive investigation into the performance of object detection models tailored for edge devices, particularly in the context of Unmanned Aerial Vehicle swarms. Object detection plays a pivotal role in enhancing autonomous navigation, situational awareness, and target tracking capabilities within UAV swarms, where computing resources are constrained by the onboard low-cost computers. Initially, a thorough review of the existing literature was conducted to identify state-of-the-art object detection models suitable for deployment on edge devices. These models are evaluated based on their speed, accuracy, and efficiency, with a focus on real-time inference capabilities crucial for Unmanned Aerial Vehicle applications. Following the literature review, selected models undergo empirical validation through custom training using the Vision Meets Drone dataset, a widely recognized dataset for Unmanned Aerial Vehicle-based object detection tasks. Performance metrics such as mean average precision, inference speed, and resource utilization were measured and compared across different models. Lastly, the study extended its analysis beyond traditional object detection to explore the efficacy of instance segmentation and proposed an optimization to an object tracking technique within the context of unmanned Aerial Vehicles. Instance segmentation offers finer-grained object delineation, enabling more precise target or landmark identification and tracking, albeit at higher resource usage and higher inference time.

Keywords: UAV; computer vision; AI; edge computing; jetson; object detection; instance segmentation; YOLO; benchmark

check for updates

Academic Editor: Dan Zhang

Received: 3 June 2025
Revised: 31 July 2025
Accepted: 1 August 2025
Published: 4 August 2025

Citation: Meimetis, D.; Daramouskas, I.; Patrinopoulou, N.; Lappas, V.; Kostopoulos, V. Comparative Analysis of Object Detection Models for Edge Devices in UAV Swarms. *Machines* **2025**, *13*, 684. https://doi.org/10.3390/machines13080684

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

In recent years, Unmanned Aerial Vehicles (UAVs) have emerged as critical tools in a wide range of applications [1], including surveillance [2], search and rescue [3], environmental monitoring [4], and precision agriculture [5]. The use of UAV swarms has further expanded the operational capabilities of these systems, enabling tasks to be completed more efficiently and robustly. However, the deployment of advanced computer vision algorithms [6], such as object detection and instance segmentation, on these platforms remains a significant challenge due to the limited computational resources available on edge devices typically used in UAVs.

The widespread adoption of UAVs in both civilian and military domains [7,8] has accelerated the demand for intelligent, autonomous systems capable of operating in dynamic

and complex environments. From border surveillance and disaster response to agricultural monitoring and infrastructure inspection, UAVs are increasingly relied upon to perform tasks that require situational awareness and decision-making [9] with minimal human intervention. In this context, computer vision technologies and especially object detection [10] are indispensable for enabling UAVs to perceive and interact with their surroundings. Surveillance, in particular, demands the ability to identify and track multiple objects in real time, which places significant computational demands on the onboard systems.

Object detection is fundamental to autonomous UAV operations, underpinning core functionalities such as obstacle avoidance, target recognition, and situational awareness. Ensuring real-time performance with high accuracy on resource-constrained hardware [11] is essential for effective deployment in dynamic environments. As such, identifying and optimizing lightweight yet powerful object detection models, as seen by the Authors in [12–15], is crucial for advancing the autonomy and effectiveness of UAV swarms.

Traditional centralized processing approaches, where raw video feeds are transmitted to a ground station or cloud server for analysis, are often impractical due to bandwidth limitations, latency concerns, and potential communication failures. As a result, edge computing has emerged as a compelling alternative, offering the benefits of low-latency inference, reduced network dependency, and improved system resilience. Real-time object detectors such as the You Only Look Once (YOLO) family of models have played a key role in this shift, starting from the first implementation as seen by authors in [16], which paved the way for further research. In continuation, authors in [17] further improved the algorithm and third-party implementations, such as [18], made improvements, and greatly enhanced the adoption of this algorithm. Recent advances in both algorithm efficiency and edge hardware performance have made it feasible to deploy powerful detectors on low-cost devices like the NVIDIA Jetson Nano, allowing UAVs to execute complex vision tasks independently and in real time. This evolution supports greater scalability and robustness in UAV swarm operations, paving the way for more autonomous and intelligent aerial systems.

Several recent studies align with the objective of this work, which is to compare and evaluate object detection models, particularly for deployment on edge devices. One comprehensive study surveys lightweight deep learning-based object detectors, emphasizing the architectural choices, training and inference challenges, and real-world deployment scenarios on datasets like MS-COCO and PASCAL-VOC, thereby underscoring the growing demand for high-performance yet low-latency models suitable for constrained environments [19]. Similarly, another review investigates simplification techniques aimed at reducing model size and computational overhead, including backbone compression and RPN replacement, highlighting the critical trade-off between speed and accuracy in edge applications [20]. In an effort more tailored to resource-constrained hardware, one paper proposed a novel Convolutional Neural Network (CNN) architecture optimized for devices like Raspberry Pi by drastically reducing input resolution while maintaining object detection accuracy, offering a practical design strategy for real-time edge deployment [21]. Broader surveys of object detection advances also explore backbone architectures and benchmark performance comparisons, providing foundational context for understanding trends in lightweight recognition models [22,23]. Complementing these studies, a recent survey focuses on lightweight deep learning models across classification, detection, and segmentation tasks and provides empirical performance evaluations on edge hardware, such as the Jetson Orin, reinforcing the importance of balancing model efficiency and accuracy in practical settings [24].

This study aimed to address this challenge by evaluating state-of-the-art object detection models optimized for edge devices, with a specific focus on their applicability

to UAV swarm systems. Through both a comprehensive literature review and empirical testing using the Vision Meets Drone (VisDrone) [25] dataset, this work assessed the trade-offs between inference speed, accuracy, and computational efficiency on the latest and most popular Jetson nano SUPER from NVIDIA, as well as the Iris Xe Graphics from Intel. Furthermore, it explored the potential of instance segmentation as an enhancement over traditional object detection, offering improved precision in complex environments, and finally, we presented an improvement on one of the most widely adopted trackers with re-identification capabilities, which modernizes its encoder inference pipeline and provides significant improvement in speed. The insights gained from this study are intended to guide future research and practical implementations of vision-based autonomy in UAV systems.

Through our approach, we addressed existing gaps present in current research in numerous ways. While prior studies have evaluated lightweight object detectors for edge devices or proposed optimizations for specific models, there remains a lack of comprehensive benchmarking tailored to modern UAV scenarios, especially involving latest-generation models, TensorRT optimization, and varying power modes. This study fills these gaps by offering holistic performance analysis across both NVIDIA Jetson and Intel Iris Xe platforms using an aerial dataset. To our knowledge, no previous study has also made an efficiency (speed/power) comparison with newer SOTA models and hardware such as the Jetson Nano SUPER and Iris Xe Graphics. In addition, we have also included the outcome of modifying training parameters for the aerial dataset and provided guidelines for other researchers. Previous studies, while they may offer a deeper description of the underlying architecture of each model, have not extended their research to metrics outside of the already provided MS-COCO results provided by their original authors.

The main contributions of the paper are as follows:

1. A comprehensive benchmarking of state-of-the-art YOLO models on Jetson Nano SUPER and Intel Iris Xe platforms. Through this effort, we provide empirical data highlighting the strengths and limitations of each model, validating their effectiveness in UAV-relevant scenarios and demonstrating their practical applicability on resource-constrained edge devices.
2. An empirical evaluation of TensorRT FP16 optimization, demonstrating its impact on both performance and energy efficiency. Our results provide concrete evidence of the benefits of low-precision inference for real-time edge deployment, supporting informed design decisions for energy-aware AI systems. At times, we were able to have an almost 4× gain in efficiency.
3. A detailed investigation into the effects of input resolution scaling on detection accuracy and inference latency. By analyzing performance trends on the VisDrone dataset, we offer guidance on selecting appropriate input dimensions to balance accuracy and speed in aerial vision applications. Our findings show smaller models, such as the YOLOv12-S, easily winning over their bigger counterparts when adjusting the tensor input.
4. A comparative evaluation of object detection and instance segmentation modes on edge platforms. This analysis quantifies the trade-offs between computational cost and perceptual richness, informing the selection of vision tasks to enhance situational awareness in UAV operations.
5. A modernization of a legacy tracking encoder's inference pipeline through the deprecation of TensorFlow V1 in favor of a lightweight, hardware-friendly implementation. This redesign yields substantial improvements in inference speed, deployment simplicity, and cross-platform compatibility, thereby enabling more practical integration into edge-based tracking systems.

The rest of the work is structured as follows: Section 2 provides a detailed rundown of the hardware and software stack used to gather results, as well as a breakdown of the available object detection models and datasets, of which some will be selected for further testing. Moreover, training parameters are mentioned. Section 3 presents the results through metrics gathered for comparing efficiency, speed, accuracy, and power consumption of all models. In addition, we present scaling results that will aid researchers in making the right choice across different models. Finally, in Section 4, we conclude our findings and bring the entire study to a close, as we also outline future approaches for further research.

2. Materials and Methods

This paper aimed to study the applicability of modern object detection methods using an aerial dataset and edge devices. Selected models were trained on the VisDrone dataset and assessed for accuracy (mAP), inference speed, and resource usage. This study also explored instance segmentation and optimized object tracking techniques to enhance target identification within UAV swarms. This section describes the approach we used to gather the results presented in Section 3.

2.1. Hardware Configuration

Experiments were conducted across multiple platforms as illustrated in Figure 1 to evaluate object detection models under both high-performance and resource-constrained environments. The hardware can be broken down into three sections:

1. For model training under accelerated conditions, experiments were conducted using an NVIDIA GeForce RTX 2080 Graphics Processing Unit (GPU) (NVIDIA Corporation, Santa Clar, CA, USA). Based on NVIDIA's Turing architecture, this discrete GPU includes 2944 CUDA cores and 8 GB of GDDR6 memory, enabling efficient parallel processing of large datasets. It features both Tensor Cores and RT Cores, which allow for mixed-precision matrix computations and real-time ray tracing, respectively. The RTX 2080 is compatible with key software frameworks such as CUDA, cuDNN, and TensorRT and integrates seamlessly with deep learning platforms like TensorFlow and PyTorch for rapid prototyping and high-throughput training.
2. The system used for initial development and debugging was equipped with an Intel Core i9-13900H processor (Intel Corporation, Santa Clar, CA, USA). This high-performance Central Processing Unit (CPU) features a hybrid architecture consisting of 14 cores (6 Performance-cores and 8 Efficient-cores) and supports up to 20 threads. It offers a maximum turbo frequency of up to 5.4 GHz and includes 24 MB of Intel Smart Cache. Fabricated using Intel 7 process technology, it is well suited for demanding computational tasks. The integrated graphics unit is the Intel Iris Xe, which provides up to 96 execution units and supports modern graphics APIs such as DirectX 12 and OpenCL 3.0. While not as powerful as a discrete GPU, the Iris Xe is useful for lightweight inference and graphical debugging workflows.
3. To evaluate performance in resource-constrained environments, inference tests were carried out on the NVIDIA Jetson Nano Super and Iris Xe Graphics (Intel Corporation, Santa Clar, CA, USA). The NVIDIA Jetson device is a compact AI computing device built for edge deployment, featuring a quad-core Arm Cortex-A78AE CPU and a 1024-core NVIDIA Ampere GPU. It is equipped with 8 GB of LPDDR5 RAM and operates within a power envelope of 7 to 25 watts, making it ideal for energy-efficient deep learning applications. The Jetson Nano Super runs on the Ubuntu-based JetPack SDK, which includes support for hardware-accelerated libraries such as TensorRT and DeepStream, as well as compatibility with many standard AI frameworks. Results

were gathered by performing inference on video streams with an input resolution of 1280 × 720 pixels. Inference times where the average of all frametimes for each run and power consumption was measured using built-in power sensors such as the VDD_IN rail on the Jetson and Package Power on the Xe GPU.

Figure 1. Hardware devices used to perform benchmarking of object detection models.

2.2. Software Environment

A diverse set of software tools and frameworks was employed to support training, inference, optimization, and evaluation workflows. The operating systems used included Windows 11 (for desktop development and training) and Ubuntu-based JetPack 6.2 (on Jetson Nano). The primary deep learning framework was PyTorch, chosen for its flexibility and large model ecosystem. To facilitate deployment across devices, models were converted to Open Neural Network Exchange (ONNX) and OpenVINO formats. For performance optimization on the Jetson Nano, TensorRT was used to accelerate inference.

The versions of the primary software stack can be seen in Table 1.

Table 1. Software versions present across the development, training and inference machines.

Component	RTX 2080 (Training)	i9-13900H (Development/Inference)	Jetson Nano (Inference)
OS/Platform	Windows 11	Windows 11	JetPack 6.2
CUDA version	12.4	—	12.6
cuDNN version	9	—	9
PyTorch version	2.6	2.7	2.5.0
Ultralytics version	8.3.141	8.3.82	8.3.82
Python version	3.10	3.10	3.10
OpenVINO version	—	2025.1	—
IPEX version	—	2.7.10 + xpu	—
ONNX version	—	—	1.17.0
TensorRT version	—	—	10.7.0

2.3. Datasets

The evaluation process utilized a combination of well-established benchmark datasets to ensure both generalizability and application-specific relevance:

- Common Objects in Context (MS-COCO): This dataset [26] was used to evaluate out-of-the-box object detection models, as it is the most widely adopted benchmark for validating model performance across a broad range of object categories and complex scenes. It is also important to note that MS-COCO captures images usually from eye level, unlike VisDrone, which is closer to a bird's eye view. Figure 2 presents example images from this dataset.
- VisDrone: After identifying the best-performing models on MS-COCO, VisDrone was used to retrain and fine-tune these models in order to showcase their effectiveness in

aerial imagery. This dataset was chosen for its relevance to real-world drone-based applications and edge computing scenarios. Figure 3 showcases the differences in point of view VisDrone has over MS-COCO.

Figure 2. Image examples from the MS-COCO dataset.

Figure 3. Image examples from the VisDrone dataset.

The MS-COCO dataset is one of the most widely used benchmarks for evaluating object detection models in diverse, real-world environments. It contains over 200,000 labeled images with more than 80 object categories, offering dense annotations including object bounding boxes, segmentation masks, and contextual information. Due to its scale and variety, MS-COCO is commonly used to validate the out-of-the-box performance of detection models before adapting them to more specialized tasks. Its comprehensive coverage of complex scenes makes it an essential first step in model selection and benchmarking.

The Vision Meets Drone (VisDrone2019) Object Detection in Images dataset, released as part of the 15th European Conference on Computer Vision (ECCV), was developed to help close the gap between current object detection capabilities and practical real-world applications. This large-scale dataset comprises 8599 aerial images captured by drones, divided into 6471 training images, 548 validation images, and 1580 testing images. Each image is richly annotated with information such as object bounding boxes, category labels, occlusion levels, and truncation ratios. In total, approximately 540,000 bounding boxes were manually labeled, covering 10 predefined object classes.

The dataset provides detailed statistics on object instances with varying degrees of occlusion across the different subsets, enabling robust evaluation under challenging

visual conditions. Data collection was carried out using multiple drone models across 14 cities, spanning diverse geographic locations and encompassing various environmental conditions, including changes in weather, lighting, and urban density. The primary focus of the dataset is on 10 object categories: pedestrian, person, bicycle, motor vehicle, van, bus, truck, awning-tricycle, and tricycle. Certain uncommon vehicle types, such as forklifts, tankers, and specialized trucks, are present but excluded from evaluation metrics. The dataset poses significant challenges for object detection algorithms due to factors such as large-scale and positional variance, frequent occlusions, and highly cluttered backgrounds, making it a rigorous benchmark for real-world drone vision tasks.

2.4. Model Training and Replication

To ensure reproducibility and fair comparison across experiments, the training process was carried out using a consistent set of hyperparameters, with slight adjustments based on model architecture and hardware limitations. While various models were evaluated, a standardized training strategy was followed to maintain comparability. Particular emphasis was placed on tuning the input image size, mosaic augmentation, scaling factor, and the number of training epochs, as these parameters were found to have the most significant impact on detection performance, all due to the characteristics of VisDrone. Using the Ultralytics [18] inference and training suite, we measured the mAP that each model produced after training and performed validation on the VisDrone dataset. We ensured consistency across all runs and libraries by implementing and freezing the development, training and inference environment specifically for the purpose of this study. The key training parameters and their adjusted ranges are summarized in Table 2.

Table 2. Adjusted training parameters during training process of the object detection models.

Parameter	Values	Description
epochs	100–300	Total number of training epochs
imgsz	416–960	Input image size in pixels
batch	2–16	Number of images per batch
mosaic	0–0.5	Mosaic data augmentation
scale	0–0.3	Image scaling augmentation
workers	1–2	Number of data workers
warmup_epochs	3–10	Number of warmup epochs before main training begins
lr0	auto–0.1	Initial learning rate
lrf	Auto–0.05	Final learning rate
optimizer	SGD–auto	Optimization algorithm used

During the training process, several key hyperparameters were adjusted to optimize model performance. The number of epochs varied between 100 and 300; however, it was observed that increasing the epochs beyond 100 resulted in minimal improvement in the model's accuracy. This suggests that the model converged relatively early during training, likely due to the effective use of transfer learning weights from the MS-COCO dataset, which provided a strong initialization and reduced the need for extended training.

The input image size (imgsz) had a significant impact on the model's performance. Larger image sizes allowed the model to capture more detailed features, improving detection accuracy, especially for small objects. This parameter was crucial because it directly influenced the spatial resolution of the input data, affecting the model's ability to localize and classify objects accurately. Although mosaic augmentation can improve generalization by combining multiple images during training, its effectiveness was limited in this study due to the presence of very small bounding boxes (less than 10 pixels). In such cases, mosaic

augmentation sometimes introduced noise rather than useful information, as the model struggled to learn from tiny objects that effectively resembled noise. Scale augmentation, which modifies the size of images randomly during training, had a minor influence on results. While it contributed to a slight improvement in robustness, the overall effect was not as pronounced as that of image size, likely because the model already handled scale variations well due to the dataset and architecture.

The number of data loading workers did not affect model accuracy, but increased RAM consumption significantly. This indicates that while parallel data loading speeds up data feeding, it comes at the cost of higher memory usage, which should be balanced depending on available system resources. Batch size showed a negligible effect on model accuracy but influenced training speed. Larger batch sizes led to faster training by enabling more parallelism during computation, while smaller batch sizes slowed the process. Despite this speed variation, the overall performance remained consistent across different batch sizes, indicating robustness of the model to this parameter. Warmup epochs, which gradually increase the learning rate at the start of training to stabilize convergence, had a small positive effect. This was particularly beneficial due to the use of transfer learning weights, helping the model adjust smoothly to the new dataset while avoiding sudden, large updates early in training.

Models trained without transfer learning consistently underperformed compared with those initialized with pre-trained MS-COCO weights, highlighting the importance of transfer learning in achieving better accuracy and faster convergence. Finally, after testing the default optimizer, the learning rates were found to be optimal. Alternative optimizers or learning rate schedules did not yield significant improvements, suggesting that these settings provided a well-balanced trade-off between convergence speed and model stability.

2.5. Preliminary Model Performance Breakdown

A large number of object detectors have been aggregated, focusing primarily on the latest versions of the YOLO architectures developed by various creators. In addition to these, modern transformer-based models like RT-DETR [27], as well as efficient architectures utilizing lightweight backbones such as MobileNet [28] and ShuffleNet [29], were included for broader benchmarking and comparison. To facilitate consistent evaluation across model scales, all architectures were categorized into nano, small, medium, and large tiers based on their floating-point operations per second (FLOPs), following the segmentation criteria defined by their respective authors. For out-of-the-box evaluation, all models presented are using a standardized input tensor size of 640 pixels on the MS-COCO dataset. This configuration aligns with the default settings presented by the original creators of each model, ensuring consistency with published performance metrics and enabling a fair comparison across architectures in their pre-trained state. The detection models with their information can be seen in Table 3.

The results in Table 3 present a comprehensive comparison of state-of-the-art object detection models evaluated on the MS-COCO dataset, across a range of model sizes and complexities. Several key insights emerge from this comparison, which inform model selection for real-time and edge deployment scenarios.

The most notable result is the consistent performance dominance of the latest YOLOv12 series. Across all model sizes—N (Nano), S (Small), M (Medium), and L (Large)—YOLOv12 variants achieve the highest mean average precision (mAP) scores while maintaining competitive computational costs (GFLOPs). The YOLOv12-M (TURBO) model, for instance, achieves 52.5% mAP with only 59.8 GFLOPs, making it an optimal candidate for real-time applications where performance must be balanced with efficiency. This cements YOLOv12 as the current benchmark leader for real-time object detection. In

contrast, lightweight models like MobileNet SSD V1/V2 and NanoDet-Plus offer extremely low computational complexity and small parameter sizes but significantly underperform in detection accuracy, with mAP values below 35%. While their speed and compactness are advantageous, the accuracy trade-off renders them suboptimal for high-performance edge devices such as the Jetson Nano Super, which can handle more capable models without compromising real-time constraints. GELAN models, despite being designed for efficiency, consistently lag slightly behind their direct competitors in the same size class, particularly YOLOv9 and YOLOv10. This suggests that while GELAN may be efficient in design, it lacks the architectural optimizations seen in the newer YOLO variants.

Table 3. Object detection model breakdown on the MS-COCO dataset as provided by the original authors.

Detection Model	mAP	Params (M)	GFLOPS
SSD MobileNet V1 FPN [30]	29.1%	-	-
SSD MobileNet V2 FPNLite [31]	28.2%	-	-
NanoDet-Plus-m-1.5× [32]	34.1% (416)	2.44	2.97 (416)
YOLOv9-T [33]	38.3%	2.0	7.7
YOLOv10-N [34]	38.5%	2.3	6.7
YOLO11-N [17]	39.4%	2.6	6.5
Gold-YOLO-N [35]	39.6%	5.6	12.1
YOLOv12-N [36]	40.6%	2.6	6.5
YOLOv12-N (TURBO) [37]	40.4%	2.5	6
GELAN-S [33]	46.7%	7.1	26.4
YOLOv9-S [33]	46.8%	7.1	26.4
YOLOv10-S [34]	46.3%	7.2	21.6
YOLO11-S [17]	46.9%	9.4	21.5
Gold-YOLO-S [35]	45.4%	21.5	46
YOLOv12-S [36]	48%	9.3	21.4
YOLOv12-S (TURBO) [37]	47.6%	9.1	19.4
GELAN-M [33]	51.1%	20	76.3
YOLOv9-M [33]	51.4%	20	76.3
YOLOv10-M [34]	51.1%	15.4	59.1
YOLO11-M [17]	51.5%	20.1	68
Gold-YOLO-M [35]	49.8%	41.3	84.5
RT-DETR-R18 [27]	46.5%	20	60
RT-DETRv2-S [38]	48.1%	20	60
YOLOv12-M [36]	52.5%	20.2	67.5
YOLOv12-M (TURBO) [37]	52.5%	19.6	59.8
GELAN-C [33]	52.5%	25.3	102.1
YOLOv9-C [33]	53%	25.3	102.1
YOLOv10-L [34]	53.2%	24.4	120.3
YOLO11-L [17]	53.3%	25.3	86.9
Gold-YOLO-L [35]	51.8%	75.1	151.7
RT-DETR-R34 [27]	48.9%	36	100
RT-DETRv2-M [38]	51.9%	36	100
YOLOv12-L [36]	53.7%	26.4	88.9
YOLOv12-L (TURBO) [37]	53.8%	26.5	82.4

The DETR family of detection transformers was previously considered state-of-the-art, especially in the transformer-based detection domain. However, the latest results show that YOLOv12 now outperforms even RT-DETRv2, both in accuracy and efficiency, further reinforcing the strength of CNN-based architectures when optimized properly. Per the YOLOv12 authors, when compared to RT-DETRv2, YOLOv12 demonstrates both speed and efficiency advantages while maintaining similar or better accuracy. For example, YOLOv12-

S outperforms RT-DETRv2-R18 by 0.1% mAP, yet it operates 42% faster and uses just 36% of the computation and 45% of the parameters. At larger scales, YOLOv12-L achieves similar accuracy to RT-DETRv2-R50 (53.7% vs. 53.4% mAP) but with 34.6% fewer FLOPs, 37.1% fewer parameters, and faster inference latency.

Given these findings, the next phase of experimentation and deployment will focus exclusively on M-sized models and below, where a balance of performance and computational feasibility is critical. This size class achieves real-time inference on edge devices while maintaining competitive accuracy, particularly with models like YOLOv12-M and YOLOv12-N (TURBO), which are emerging as the most promising candidates for robust, real-time object detection in constrained environments.

2.6. Instance Segmentation and Tracking

To ensure an even and fair comparison between object detection and instance segmentation, we used the same base models for both tasks. Specifically, we evaluated YOLOv11-N, YOLOv11-S, and YOLOv11-M, along with their corresponding instance segmentation extensions. By doing so, we isolated the performance overhead introduced solely by the segmentation component without introducing variability from different architectures or model sizes. Both detection-only and segmentation-capable models share identical backbones and feature extraction layers, guaranteeing that any differences in inference time reflect the additional computation required to generate pixel-level masks rather than disparities in base model complexity.

All models were tested in two hardware environments to provide a comprehensive view of performance trade-offs. On Intel Iris Xe, we evaluated the standard YOLOv11 implementations converted to OpenVINO format, while on NVIDIA Jetson, we employed TensorRT-optimized YOLOv11 models with FP16 precision for accelerated inference. In both settings, we measured and compared the inference latency for detection-only and instance segmentation modes. This methodology allowed us to quantify the relative increase in execution time attributable to instance segmentation across different model scales and hardware platforms. To finalize this study, we concluded our contributions by focusing on improving and evaluating multiple object tracking performance by altering only the encoder inference pipeline, as seen in Figure 4 within the DeepSORT [39] framework, while keeping the detection model unchanged. This design ensured that any observed differences in execution time arose solely from changes to the encoder and not from variations in detection output. We tested three encoder configurations: the original TensorFlow implementation, a reengineered PyTorch version with built-in batching, and a newly integrated ONNX version. All tracking tests were driven by detections produced by our VisDrone-trained model, allowing us to directly assess the effect of encoder modifications on tracking speed and stability.

The TensorFlow V1 encoder model was first converted to PyTorch and ONNX formats to enable these comparisons. We updated the DeepSORT codebase as needed, including revisions to the ImageEncoder class to accommodate the new inference engines and the batched feature extraction logic. Tracking latency was measured on scenes with dense object populations, recording both frame-level processing time and the number of active tracklets. This methodology allowed us to clearly identify the efficiency gains from batching and to evaluate the suitability of each encoder design for real-time tracking under challenging conditions.

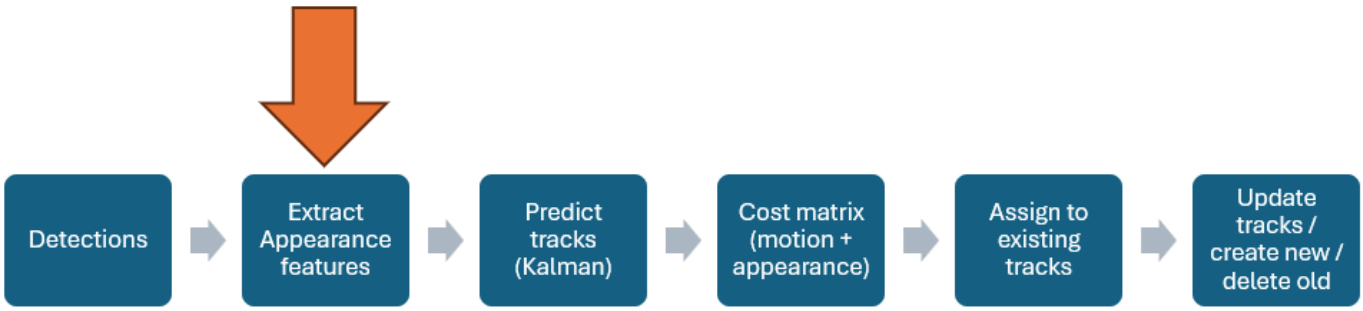

Figure 4. DeepSORT execution time in a congested scene using different encoder inference pipelines. Initially, the encoder used in DeepSORT follows the steps presented in Algorithm 1. Batching takes place by determining how many detection crops will be included in the process, and for our testing, we had it set to 128, which allows inferencing all detections in a single "pass". Given that TensorFlow V1 is deprecated, we used V2 in compatibility mode.

Algorithm 1. Default code behavior for Tensorflow DeepSORT encoder inference

1. Import TensorFlow and configure GPU memory limits (e.g., limit to 2 GB).
2. Load the encoder model using 'tf.saved_model.load()'.
3. Access the default inference signature ('serving_default').
4. Retrieve input and output tensor names and shapes.
5. For each frame:
    a. Convert detection crops to TensorFlow tensors.
    b. Pass tensors through the model using the signature.
    c. Extract output features as numpy arrays.
    d. Measure and record inference time for analysis.

Algorithm 2 describes the refactoring that allowed us to enable inference using the ONNX runtime. The TensorFlow weights were converted to ONNX while maintaining the same shape. Similar to TensorFlow, ONNX handles batching internally, and we had our batch size set to the same number of 128 detections.

Algorithm 2. Refactoring of the DeepSORT encoder inference pipeline using ONNX

1. Initialize an ONNX Runtime session with GPU and CPU execution providers.
2. Load the ONNX model file and retrieve input/output names and shapes.
3. For each frame:
    a. Format detection crops as input arrays.
    b. Run inference using 'session.run()' with input dictionary.
    c. Extract feature outputs from the result list.
    d. (Optional) Post-process features if needed for downstream tasks.

Finally, Algorithm 3 specifies how we achieved inference using torch. Initially, the ONNX model is loaded and then converted to PyTorch for inference. We used the torch.stack() function to generate batches and then pass them for inference.

Algorithm 3. Refactoring of the DeepSORT encoder inference pipeline using PyTorch

1. Load the ONNX model using 'onnx.load()'.
2. Convert the ONNX model to PyTorch using 'ConvertModel'.
3. Move the model to GPU and set to evaluation mode.
4. For each frame:
    a. Stack detection crops into a PyTorch tensor and transfer to GPU.
    b. Divide inputs into batches (e.g., batch size = 128).
    c. For each batch:
        i. Run inference
        ii. Move output features to CPU and convert to numpy.
    d. Concatenate batch outputs for downstream processing.

3. Results

Having described the Materials and Methods in Section 2, the analysis of our findings takes place in Section 3. Due to the limited resources found in edge devices, we pay special attention to power consumption, accuracy, and inference speed across our findings. While there is no hard limit on power consumption, the lower it is, the better, as the edge devices are usually battery powered, especially in UAV scenarios. Accuracy wise, the same remains true; the higher the mAP we can achieve, the better, while we must remain within the frame time window of 33.3 ms for real-time processing. This hard limit is derived from the fact that cameras usually output at 30 FPS. This ensures that every frame is processed in time and no information is lost from dropped frames.

3.1. Baseline Model Characteristics on the Jetson Nano SUPER

To quantify the feasibility of deployment and the efficiency of the models discussed in Section 2.5 for real-time applications on edge devices, we conducted a series of benchmarks measuring inference speed and power consumption using the NVIDIA Jetson Nano SUPER. These evaluations were performed under three distinct power modes, as detailed in Table 3: M1 (15 W), M2 (25 W), and M3 (MAX_N), the latter being a newly introduced maximum performance mode specific to the Jetson Nano SUPER.

All benchmarks were conducted with jetson_clocks enabled to ensure consistency and accuracy in performance measurements. The jetson_clocks utility is an NVIDIA command-line tool that locks the GPU, CPU, and memory frequencies to their maximum values, thereby eliminating variability due to dynamic frequency scaling and ensuring a stable benchmarking environment. Power consumption was measured directly from the VDD_IN rail, which supplies the primary input voltage to the Jetson module. The video stream used for model inference had a resolution of 1280 $\times$ 720 pixels. Speed and Power results were gathered by averaging out the per-frame results from each test session. The results are presented in Table 4. M3 power mode overcomes a lot of the inefficiencies present in the out-of-the-box performance of the YOLO models when running on the Jetson. Models that previously failed to achieve real-time inference can now run below 33.3 ms and pass the threshold. This topic will be studied in more detail in the following sections, where we enable TensorRT optimizations.

Table 4. Out-of-the-box inference performance and power consumption on NVIDIA Jetson for SOTA object detection models.

Detection Model	M1 Speed (ms)	M1 Power (w)	M2 Speed (ms)	M2 Power (w)	M3 Speed (ms)	M3 Power (w)
YOLOv9-T [33]	48.7	7.3	52.7	6.9	40.6	8.9
YOLOv10-N [34]	23.9	7.8	26.4	7.8	23.1	8.2
YOLO11-N [17]	24.5	7.8	27.1	7.1	22.4	8.3
YOLOv12-N [36]	37.5	7.8	40.8	7.1	32.6	8.4
GELAN-S [33]	49.6	8.7	56.5	8.0	42.4	10.4
YOLOv9-S [33]	49.4	8.7	56.3	8.1	42.4	10.4
YOLOv10-S [34]	24.6	10.1	26.5	8.9	24.5	10.5
YOLO11-S [17]	25.2	10.2	31.6	8.9	24.7	10.3
YOLOv12-S [36]	37.8	9.5	42.4	8.8	32.7	10.7
GELAN-M [33]	57.4	11.6	41.7	14	37.0	16.1
YOLOv9-M [33]	57.3	11.5	41.6	14	37.1	15.9
YOLOv10-M [34]	42.8	11.7	33.2	13.0	28	15.1
YOLO11-M [17]	44.4	11.7	33.1	13.6	29.3	15.2
YOLOv12-M [36]	56.2	11.8	42.7	13.8	37.1	15.9

Model performance wise, GELAN and YOLOv9 models exhibit very similar behavior across all scales and power modes, reflecting underlying architectural similarities and consistent performance characteristics. Notably, although YOLOv12 is expected to have similar execution time to its predecessors while being more accurate due to architectural changes, such as the integration of the Flash-Attention mechanism, our results show degraded out-of-the-box performance when compared to YOLOv11. We envision that this larger-than-expected drop in inference speed is attributed to limited resources being present when it comes to memory bandwidth and compute units. A detailed breakdown of the architectural difference between the two latest versions of YOLO is seen in Table 5. It is likely that the NVIDIA T4 Tensor GPU, which is widely used for validation, hides some of the impact due to its larger capabilities.

Table 5. Architecture breakdown changes of YOLOv11 and YOLOv12.

Aspect	YOLOv11	YOLOv12
Architecture style	CNN-based design with C3K2 modules and depthwise separable head	Attention-centric design incorporating Area Attention and R-ELAN
Backbone	Stacked C3K2 (GELAN variant) blocks	Area Attention backbone with a simplified final stage using a single R-ELAN block
Feature aggregation	ELAN/GELAN aggregation without residual connections	R-ELAN aggregation with residual connections for improved stability and efficiency
Attention	No attention mechanism; fully convolutional	Area Attention with Flash-Attention for efficient global modeling
Positional encoding	Implicit through convolutional locality	No explicit positional encoding; employs large separable convolution (7×7) for positional perception
Normalization	Convolution with batch normalization	Convolution with batch normalization in place of typical transformer layer normalization
Block stacking	Multiple C3K2 blocks in final stage	Single R-ELAN block in final stage for simplified optimization

One of the key observations is that M2 mode performs unexpectedly worse than M1 for many of the smaller models (e.g., YOLOv9-T, YOLOv10-N, YOLO11-N). This counterintuitive behavior, where M2 delivers slower inference despite similar or lower power consumption, can likely be attributed to NVIDIA's internal power and thermal

management strategies. In this mode, frequency scaling or thermal throttling may become more aggressive due to lower power ceilings, resulting in inconsistent GPU utilization and degraded performance, especially when the models are lightweight.

On the other hand, performance scales more predictably in larger models (e.g., GELAN-M, YOLOv10-M), with noticeable improvements from M1 to M3 in both inference speed and efficiency. This trend suggests that power modes have a stronger positive impact as GPU occupancy increases, allowing the hardware to better leverage the extra thermal and power headroom. In M3 (MAX_N), which is exclusive to the Jetson Nano SUPER, the models consistently achieve their fastest inference times, demonstrating its importance for real-time edge applications.

Another noteworthy observation from Table 4 is the relatively small gap in inference speed between the small and large model variants under the M3 (MAX_N) power mode. For instance, while lightweight models like YOLOv10-N and YOLO11-N achieve inference times of 23.1 ms and 22.4 ms, respectively, the medium-sized versions of the same architectures (YOLOv10-M and YOLO11-M) run at only slightly slower speeds of 28 ms and 29.3 ms, respectively. Similarly, GELAN-S and YOLOv9-S both record 42.4 ms, which is nearly identical to their medium counterparts, GELAN-M and YOLOv9-M, which come in at 37.0 ms and 37.1 ms. This narrow performance differential suggests that a significant portion of the total inference time is being consumed not by the model execution itself, but by auxiliary processes, such as data transfers between the CPU and GPU or preprocessing/postprocessing overhead. These outcomes are also visualized in Figure 5. Each graph portrays a power mode with the *X*-axis consistently describing the inference time as previously seen in Table 4. The *Y*-axis shows the instant power consumption used to achieve the result. Models placed in the lower left part of the graph are better as they are the most efficient.

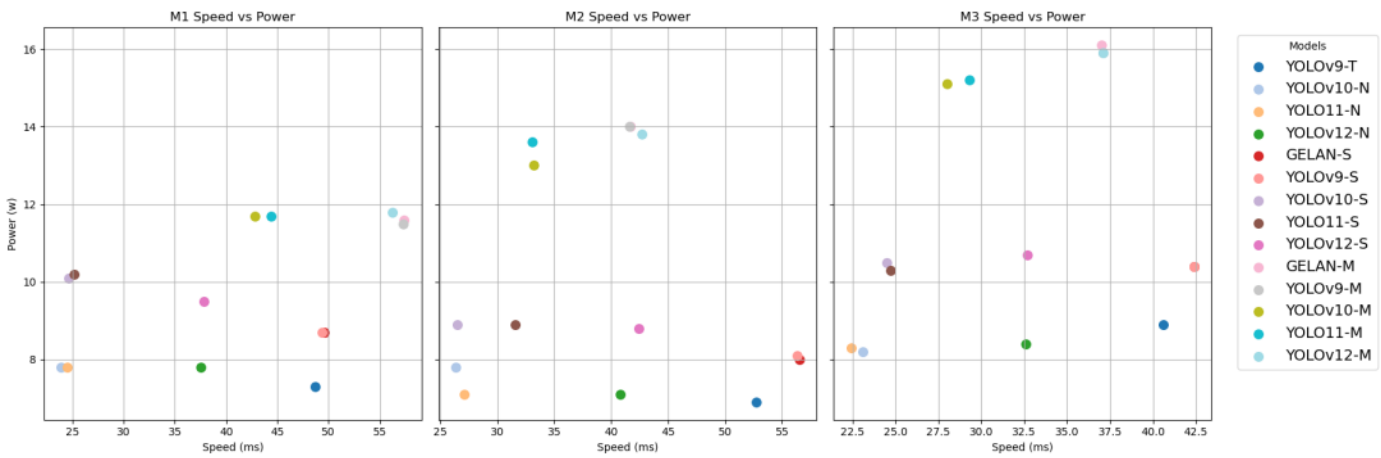

Figure 5. Speed versus power scatter chart for all different power modes.

3.2. Model Characteristics on the Iris Xe Graphics

Table 6 presents the inference performance and power consumption of several state-of-the-art object detection models when executed on Intel Iris Xe integrated graphics. For the small and medium variants of the YOLOv10, YOLOv11, and YOLOv12 families, we exported the models to OpenVINO format to enable optimized execution on Intel hardware. Inference tests were conducted using the OpenVINO Runtime to leverage hardware-specific acceleration provided by the Intel Iris Xe GPU.

Table 6. Inference performance and power consumption on Intel Iris Xe graphics for SOTA object detection models.

Detection Model	Speed (ms)	Power (w)
YOLOv10-N [34]	23.5	15.1
YOLO11-N [17]	15.6	13.6
YOLOv12-N [36]	23.1	15.5
YOLOv10-S [34]	35.6	19.3
YOLO11-S [17]	24.4	18.4
YOLOv12-S [36]	32.8	19.1
YOLOv10-M [34]	53.2	24.6
YOLO11-M [17]	48.5	24.9
YOLOv12-M [36]	65.4	25.8

From the results presented in Table 6, it is evident that Intel Iris Xe Graphics can support real-time inference ($\geq$30 FPS) for tiny and small-sized detection models when using a 640 $\times$ 640 input resolution. Notably, the YOLOv11-N model achieved the best latency at 15.6 ms, while the YOLOv10-N and YOLOv12-N models followed closely with 23.5 ms and 23.1 ms, respectively. All three models maintained inference speeds well above the real-time threshold.

For small-sized models, such as YOLOv11-S, the inference time remained efficient at 24.4 ms, enabling real-time throughput. However, performance begins to degrade with medium-sized models, where latency increases substantially, e.g., YOLOv12-M required 65.4 ms per frame, which corresponds to approximately 15 FPS, below real-time performance.

While the Iris Xe GPU demonstrates competitive inference speeds for tiny and small models, it does so at the cost of higher power consumption. For example, YOLOv10-N consumed 15.1 W compared with only around 7–9 W typically observed on the Jetson platform for similar workloads. This trend continues in the small and medium categories, where Iris Xe models draw roughly twice the power of their Jetson counterparts.

It is important to note that all models executed on Iris Xe were exported to OpenVINO IR format, enabling hardware-specific optimization, whereas Jetson benchmarks were performed using stock FP32 PyTorch models. Despite this optimization, the Jetson platform maintains a clear advantage in medium-sized models, offering both better performance and energy efficiency.

3.3. Model Optimization for Edge

In this updated evaluation, the previously tested models originally benchmarked using out-of-the-box FP32 PyTorch implementations are now compared to their optimized counterparts exported to TensorRT at FP16 precision. This transition leverages TensorRT's capabilities for acceleration and quantization, specifically targeting reduced latency and improved efficiency. Importantly, the power configuration remains unchanged throughout testing: all benchmarks were conducted with the Jetson device set to MAX_N power mode and jetson_clocks enabled to ensure maximum performance and consistent power delivery across both FP32 and FP16 runs. This controlled environment allows for a fair and direct comparison of inference performance and energy efficiency between the native PyTorch models and their TensorRT-optimized versions.

The benchmarking results in Table 7 clearly demonstrate the impact of TensorRT optimization and model size on inference speed and power efficiency across the YOLOv10, YOLO11, and YOLOv12 series. A standout observation is how much faster the Nano (N) models perform after conversion to FP16 TensorRT. For instance, YOLOv10-N sees its inference time drop from 23.1 ms to 13.0 ms, representing a 43.7% speedup. Simi-

larly, YOLO11-N and YOLOv12-N show speedups of 45.5% and 36.5%, respectively. This dramatic improvement is largely attributed to the efficiency of TensorRT's optimization pipeline, which is particularly effective on smaller, streamlined architectures like the Nano variants. As FP16 allows efficient usage of the tensor cores, we now see that all models can now run at below 33.3 ms, which now allows the usage of the SOTA YOLOv12-M model with the Jetson.

Table 7. TensorRT model optimization for FP16 inference using NVIDIA Jetson Nano SUPER.

Detection Model	FP32 Speed (ms)	FP32 Power (w)	FP16 TensorRT Speed (ms)	FP16 TensorRT Power (w)
YOLOv10-N [34]	23.1	8.2	13.0	6.8
YOLO11-N [17]	22.4	8.3	12.2	6.9
YOLOv12-N [36]	32.6	8.4	20.7	6.8
YOLOv10-S [34]	24.5	10.5	21	6.9
YOLO11-S [17]	24.7	10.3	19.9	7
YOLOv12-S [36]	32.7	10.7	26.7	7.7
YOLOv10-M [34]	28	15.1	25.9	8.5
YOLO11-M [17]	29.3	15.2	29.4	8.1
YOLOv12-M [36]	37.1	15.9	32.3	10.1

A visual representation of the gains can be seen in Figure 6. The *Y*-axis on the first graph portrays inference speed in ms (lower is better), and on the second graph, instant power consumption. The respective columns describe the same model tested on both graphs according to the labels on the bottom *X*-axis. After converting the Nano models to FP16 TensorRT, the energy per frame drops by over 50% in every case. For example, YOLOv10-N goes from 0.18942 J/frame to 0.0884 J/frame, a reduction of approximately 53.3%. YOLO11-N and YOLOv12-N show similar improvements of around 54.7% and 48.6%, respectively.

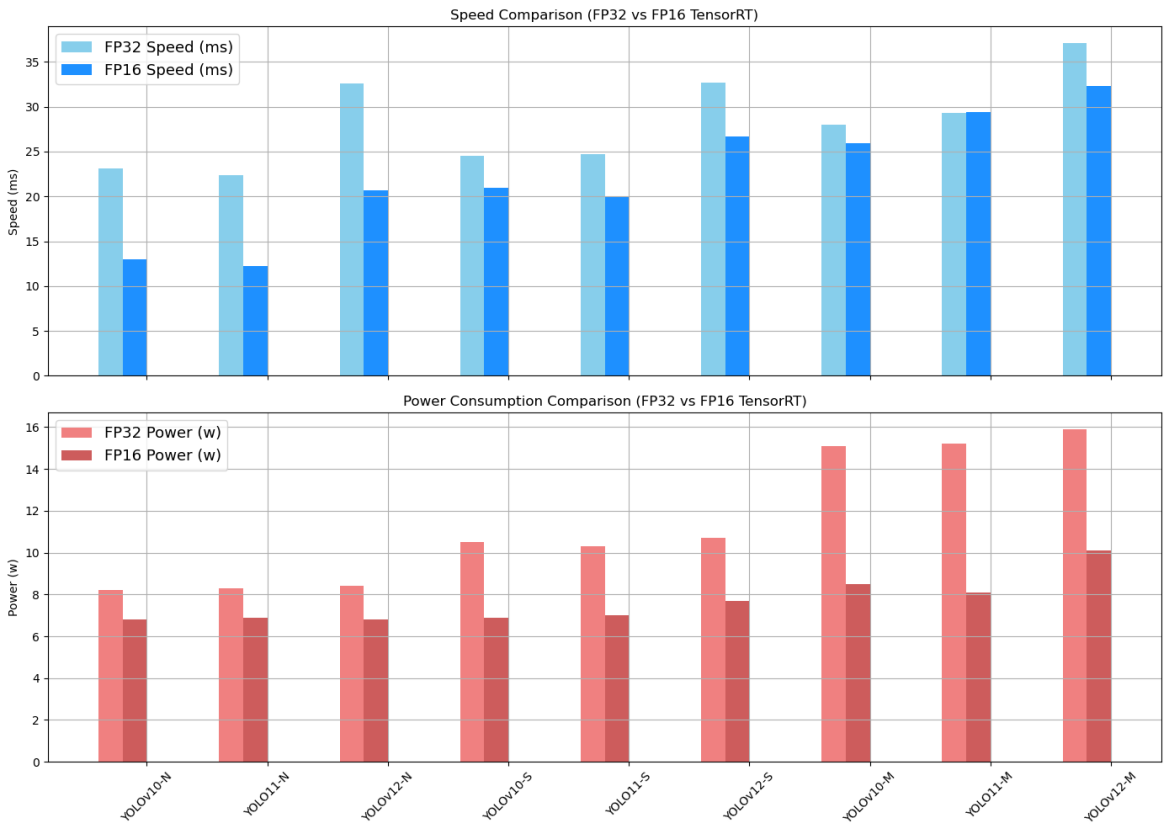

Figure 6. Performance and power consumption improvement using TensorRT.

The YOLO-M models also see significant energy savings per frame after conversion to FP16 TensorRT:

- YOLOv10-M reduces energy per frame from 0.4228 J to 0.22015 J (~48% reduction);
- YOLO11-M goes from 0.44536 J to 0.23814 J (~46.5% reduction);
- YOLOv12-M improves from 0.59049 J to 0.32623 J (~44.8% reduction).

Although the M models are inherently more computationally intensive than the N models, TensorRT optimization still nearly halves the energy cost per frame. This makes the Medium models viable in scenarios where higher accuracy is needed but power constraints still matter, such as in semi-autonomous systems, robotics, or smart surveillance.

These results confirm that TensorRT optimization not only reduces inference time but also significantly decreases the energy cost per frame, which is crucial for battery-operated or thermally constrained environments.

3.4. Results of Models Trained with VisDrone

As mentioned in Section 2.4, we performed an investigation regarding how the latest YOLO models perform in the VisDrone dataset and modified training parameters to try to optimize the end result. Example training images are seen in Figure 7 where each bounding box corresponds to an annotated class object distinguished by the number and color presented. Our focus was put on the YOLOv12-S model, as it presents a middle ground between size and accuracy. We aggregated 21 training results, each with different parameters and final mAP scores, as seen in Table 7. The results were gathered on the NVIDIA Jetson unit with a tensor input of 640 px and the validation dataset of VisDrone 2019.

Figure 7. Training batch with augmentations applied using the VisDrone Dataset.

From the results in Table 8, it is clear that training strategy and input resolution significantly impact the out-of-the-box mAP performance of YOLOv12 models on NVIDIA Jetson hardware. Notably, YOLOv12-S exhibits a wide range of outcomes depending on factors like weight transfer, warmup epochs, and resolution. For example, at 416 px with weight transfer and three warmup epochs, it reaches 32.8% mAP, but the same model without weight transfer drops to 26.7%, showing a 6.1-point gap. This confirms the effectiveness of transfer learning in enhancing initial performance.

Table 8. Detection model performance across varying training configurations, highlighting the impact of image size, warmup, optimization, and weight transfer on mAP.

Detection Model	Epochs	Imgsz	Mosaic	Warmup Epochs	Weight Transfer	Optimizer	Scale	mAP
YOLOv12-N	100	416	0%	3	yes	auto	0%	25.6%
YOLOv12-S	100	416	0%	10	yes	auto	0%	30.1%
YOLOv12-S	100	416	0%	3	yes	auto	0%	32.8%
YOLOv12-S	100	416	0%	10	no	auto	0%	26.7%
YOLOv12-S	100	416	50%	3	yes	auto	0%	29.1%
YOLOv12-S	100	416	0%	3	no	auto	30%	29.1%
YOLOv12-S	300	416	0%	10	no	auto	0%	26.8%
YOLOv12-S	100	640	0%	3	yes	auto	0%	34.7%
YOLOv12-S	100	768	0%	3	yes	auto	0%	36.1%
YOLOv12-S	100	960	0%	3	yes	auto	0%	36.9%
YOLOv12-S	200	608	0%	3	yes	auto	0%	34.3%
YOLOv12-M	100	416	0%	3	yes	auto	0%	34.4%
YOLOv12-M	100	416	0%	10	yes	auto	0%	35.3%
YOLOv12-M	100	416	0%	3	no	auto	0%	28.9%
YOLOv12-M	100	416	0%	5	yes	SGD LR0_0.1	0%	34.8%
YOLOv12-M	100	416	0%	5	yes	SGD LR0_0.1 LRF_0.05	0%	34.9%
YOLOv12-M	300	416	0%	5	yes	SGD LR0_0.1 LRF_0.05	0%	35.4%
YOLOv12-L	100	416	0%	3	yes	auto	0%	38.3%
YOLOv11-N	100	416	0%	3	yes	auto	0%	26.8%
YOLOv11-S	100	416	0%	3	yes	auto	0%	31.5%
YOLOv11-M	100	416	0%	3	yes	auto	0%	36%

Input resolution also plays a major role. Increasing YOLOv12-S's image size from 416 px to 960 px boosts mAP from 32.8% to 36.9%, a 4.1-point improvement, without changing other hyperparameters. This mirrors trends seen elsewhere in the study, where moderate increases in input resolution help capture finer object details, especially in dense datasets like VisDrone. As seen in Figure 8, the presented F1 curve for the training season of YOLOv12-S with 960 px tensor input showcases that the car class achieves the highest peak F1 score (above 0.8), indicating strong precision-recall performance, followed by bus and motor. The class involving pedestrians shows worse performance due to the smaller detected objects. However, models must be well-tuned to benefit—simply training for longer (e.g., 300 epochs at 416 px) without the right settings only yielded 26.8% mAP, underperforming compared to better-configured runs with fewer epochs.

For YOLOv12-M, similar gains are observed with tuning. The base config yields 34.4%, but switching to SGD with specific learning rate scheduling increases this to 35.4% after 300 epochs. Interestingly, using SGD without proper warmup or tuning gives inconsistent results (e.g., 28.9%), again emphasizing that optimizer choice alone is not enough.

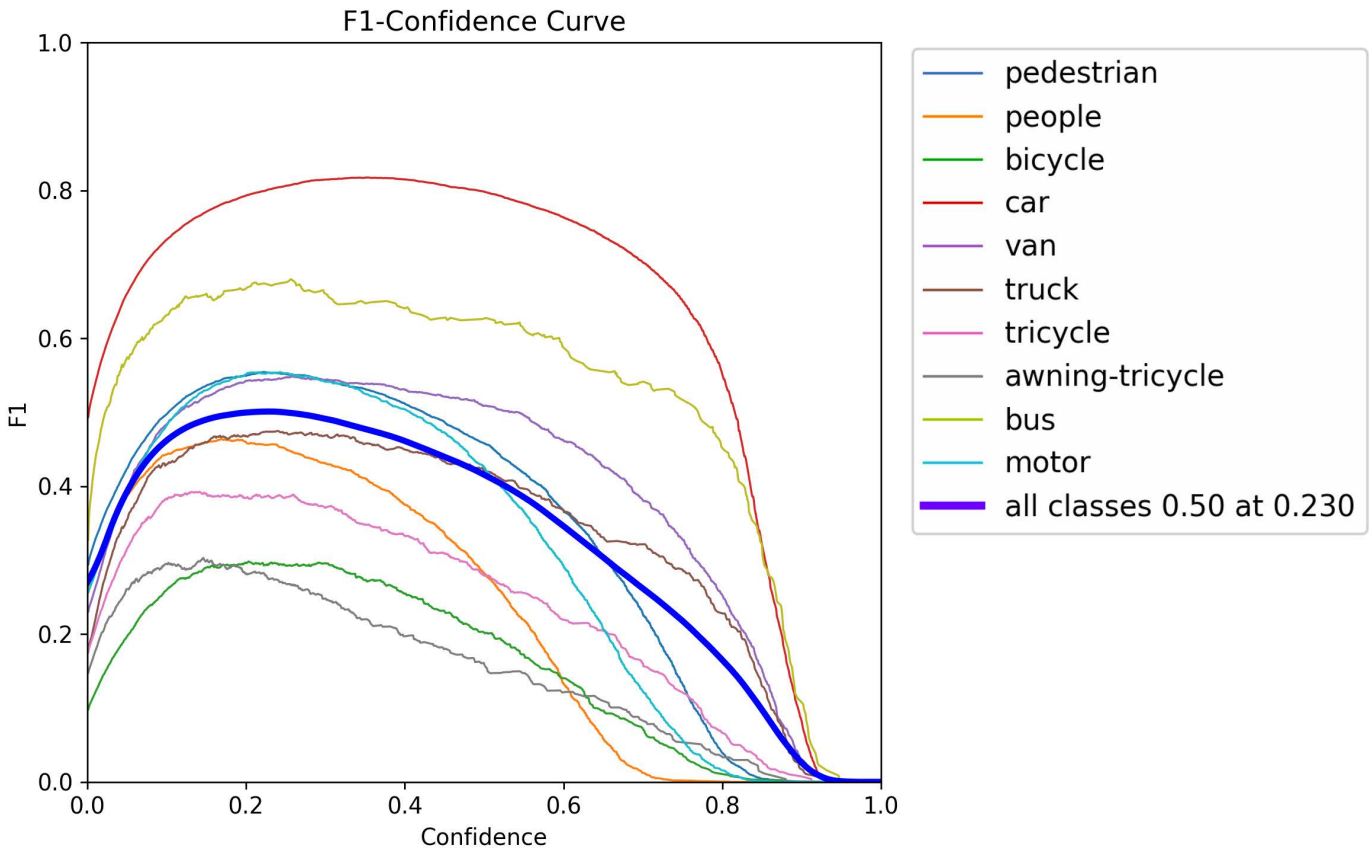

Figure 8. F1-Confidence curve of YOLOv12S training on the VisDrone dataset.

3.5. Scaling of Models with Tensor Input and Size

Due to the nature of aerial photography, it is common that inside a picture frame, there can be dozens, if not hundreds, of detections of objects, which means that high-definition imagery is required. Multiple object detection models will then downsample the image to whichever tensor input mode they are running, which results in lost information. To find the sweet spot, we ran a plethora of tests using our own custom-trained YOLO models on the VisDrone dataset.

Table 9 shows that significant gains in detection accuracy were observed when increasing the input size from 432 px to 640 px, across all YOLOv12 model variants. For example, YOLOv12-L improved from 32.7% to 38.3% mAP, a 5.6 percentage point gain, while YOLOv12-S jumped from 28.3% to 32.8%, a 4.5-point improvement. These results highlight how even a moderate increase in input resolution close to the original training resolution of 416 px can yield meaningful improvements in performance, likely due to better spatial feature resolution.

Table 9. YOLOv12 mAP across different tensor input sizes and models using the VisDrone dataset.

Detection Model	432 px	640 px	768 px	960 px
YOLOv12-N	22.3%	25.6%	26.1%	24.5%
YOLOv12-S	28.3%	32.8%	33.5%	31.9%
YOLOv12-M	31%	34.4%	36.8%	35.7%
YOLOv12-L	32.7%	38.3%	38.7%	37.2%

However, moving beyond 640 px yields diminishing returns. For instance, YOLOv12-M gains only 2.4 points going from 640 px to 768 px (from 34.4% to 36.8%) and then slightly

drops to 35.7% at 960 px. Similarly, YOLOv12-N goes from 25.6% to 26.1% at 768 px and then decreases to 24.5% at 960 px. This trend indicates that increasing the resolution beyond a certain point (especially past 640 px) does not consistently benefit detection and may even introduce performance instability, possibly due to divergence from the original 416 px training scale or higher noise at large scales.

Interestingly, the results also show that smaller models like YOLOv12-S can be competitive with larger ones under certain resolutions. At 768 px, YOLOv12-S scores 33.5%, nearly matching YOLOv12-M at 36.8% and not far behind YOLOv12-L at 38.7% despite using fewer parameters and less compute. This suggests that for resource-constrained environments, smaller models at moderately higher resolutions can offer excellent performance-to-efficiency trade-offs. In Figure 9, we present a bar chart of the results shown in Table 9. The *X*-axis splits the models per input resolution, and the *Y*-axis portrays the detection accuracy. Through this graph, we can easily distinguish how smaller models can beat larger ones just by increasing input resolution.

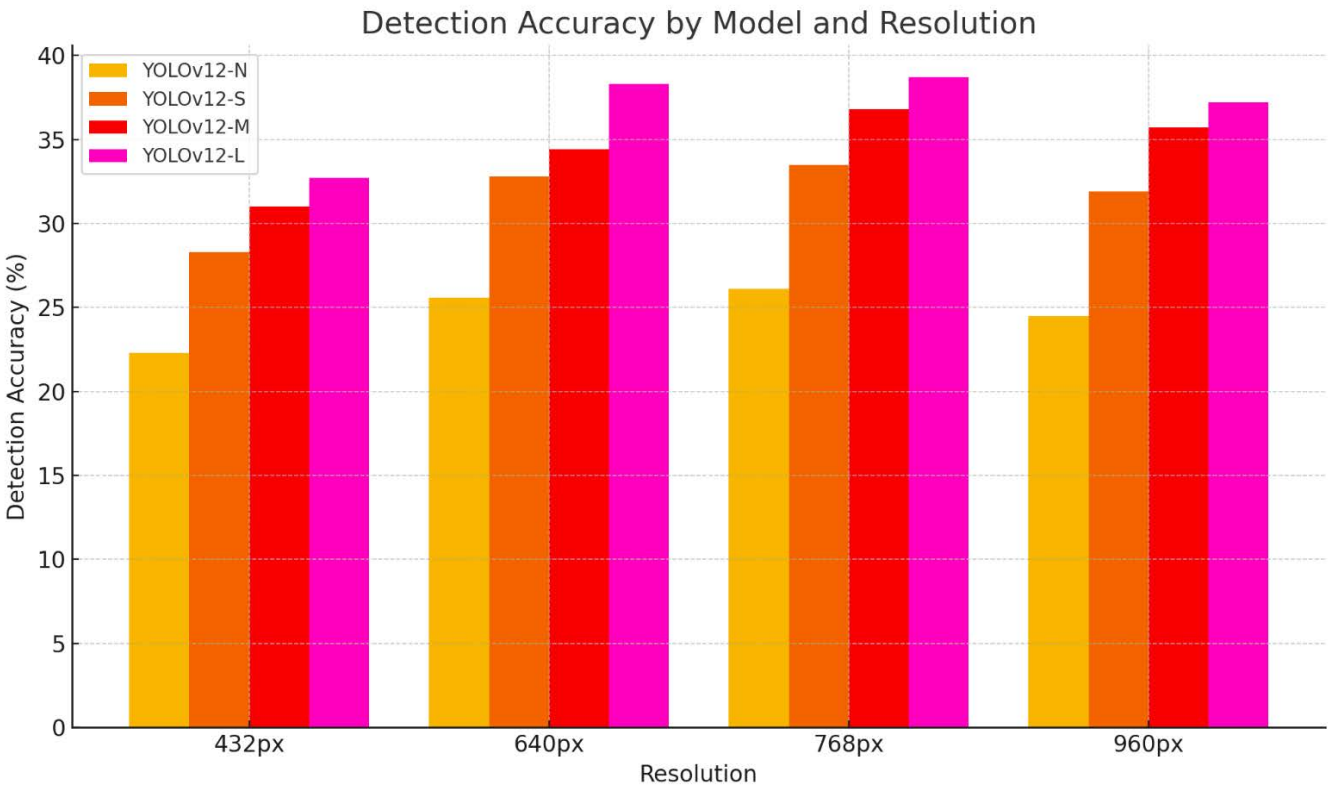

Figure 9. Model accuracy by size and resolution.

3.6. Instance Segmentation

In this section, we evaluate instance segmentation performance in addition to traditional object detection to understand the trade-offs involved when extending detection models for more granular visual understanding. Instance segmentation operates on top of the existing detection framework by leveraging the same backbone and feature extraction layers to not only localize objects with bounding boxes but also generate pixel-level masks for each instance [40]. This additional layer of semantic detail is computed using the features already extracted during the detection process, making it a natural and efficient extension. Importantly, these high-resolution features and per-instance masks can significantly enhance downstream tasks such as object tracking, where precise spatial boundaries help improve data association and robustness, especially in crowded or overlapping scenes. Being a newer method, there is not extensive literature on this topic, but authors in [41]

present InstaTrack, which is a cutting-edge method that uses instance segmentation to enhance appearance-based tracking. InstaTrack efficiently generates strong appearance features after being trained with contrastive loss and non-negativity enforcements. In order to evaluate object visibility and maximize the usage of appearance features, it computes coverage values, and the authors presented promising tracking results using MOT Metrics. Finally, SAM 2 [42], a widely adopted segmentation and tracking transformer, has proven the benefits of 'masklets', albeit using a different approach when compared to the more traditional methods studied in this paper.

Table 10 reports the inference performance of YOLOv11 object detection models on Intel Iris Xe, comparing detection-only and instance segmentation modes. Across all model variants, instance segmentation significantly increases inference time. For the lightweight YOLOv11-N, segmentation is approximately 60.9% slower than detection (25.1 ms vs. 15.6 ms). The gap is similar for YOLOv11-S, with segmentation introducing a 56.6% slowdown (38.2 ms vs. 24.4 ms). For the mid-sized YOLOv11-M model, segmentation takes 60% longer than detection (77.6 ms vs. 48.5 ms). The impact segmentation has on execution time is apparent, and only the nano variant of YOLOv11 can achieve real-time processing.

Table 10. Out-of-the-box inference performance on Iris Xe for YOLOv11 object detection and segmentation.

Detection Model	Speed_Detect (ms)	Speed_Segment (ms)
YOLOv11-N [34]	15.6	25.1
YOLO11-S [34]	24.4	38.2
YOLOv11-M [34]	48.5	77.6

Table 11 reports the inference performance of YOLOv11 object detection and instance segmentation models on the NVIDIA Jetson platform, comparing detection-only and segmentation modes using TensorRT. Across all model variants, instance segmentation introduces a noticeable increase in inference time compared with detection alone. For the lightweight YOLOv11-N, segmentation is approximately 24.6% slower than detection (15.2 ms vs. 12.2 ms). The YOLOv11-S model shows a similar pattern, with segmentation being 25.1% slower than detection (24.9 ms vs. 19.9 ms). For the mid-sized YOLOv11-M, segmentation incurs a 44.6% increase in latency (42.5 ms vs. 29.4 ms). These results highlight the additional computational cost of producing pixel-level masks and also show the increased performance drop as the models get bigger due to saturation of the hardware. For the first time, we are unable to achieve real-time processing across all models when using FP16 optimization as the YOLOv11-M model failed to meet the threshold of 33.3 ms.

Table 11. Inference performance on NVIDIA Jetson for YOLOv11 TensorRT FP16 object detection and segmentation.

Detection Model	Speed_Detect (ms)	Speed_Segment (ms)
YOLOv11-N [34]	12.2	15.2
YOLO11-S [34]	19.9	24.9
YOLOv11-M [34]	29.4	42.5

3.7. Multiple Object Tracking

Multiple object tracking is an operation closely related to multiple object detection, which we studied previously. Without accurate and fast object detection, object tracking cannot take place. Moreover, object tracking provides a clear use case for the real world, as it enables tasks like traffic monitoring, object following, and analytics by being able to associate detections between frames.

Figure 10 illustrates the tracking time per frame (left y-axis) for three implementations of the encoder inference pipeline: the original TensorFlow (TF)-based method (red), our PyTorch-based version (green), and our ONNX-based version with optimized batched inference (blue). The number of tracklets processed per frame is also shown (right y-axis, gray).

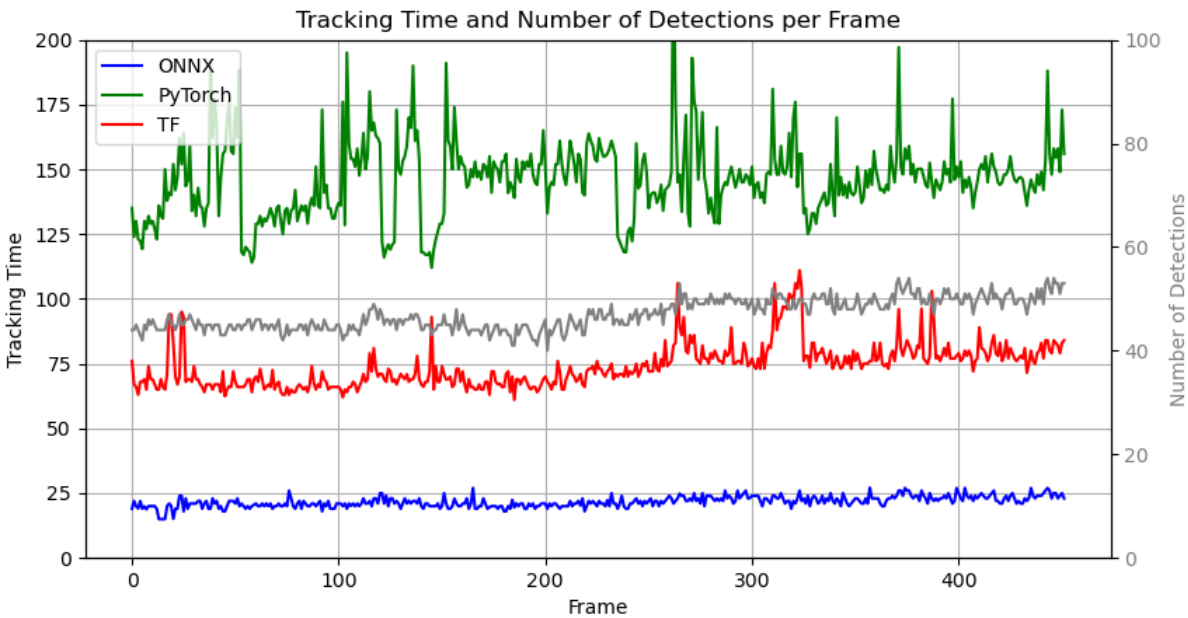

Figure 10. DeepSORT execution time in a congested scene using different encoder inference pipelines.

Our results reveal several key insights:

1. The original TensorFlow implementation consistently exhibits higher tracking times, averaging around 70–90 ms per frame, with notable spikes when the number of tracklets increases.
2. The PyTorch implementation increases the tracking time modestly, achieving unstable performance in the 130–160 ms range. This may be caused by thread sync inconsistencies between the PyTorch models for detection and tracking that are now running in series
3. The ONNX-based implementation, which incorporates a batched inference strategy, delivers the most substantial performance improvement. Tracking times remain consistently low, around 15–25 ms per frame, even as the number of tracklets increases. This demonstrates the efficiency gained by batching feature encoding requests, minimizing redundant computation and improving GPU utilization.

The gray line in the plot confirms that fluctuations in tracking time correlate with the number of tracklets processed per frame. However, the ONNX pipeline's ability to handle variable load with minimal latency highlights its suitability for real-time deployment on edge devices and in high-density tracking scenarios.

Our Contribution to DeepSORT, when paired with a lightweight Detector that we studied previously, will therefore allow real-time execution even under complex scenes.

4. Conclusions

This paper explored the feasibility and optimization of state-of-the-art object detection models deployed on edge computing platforms, specifically the NVIDIA Jetson Nano SUPER and Intel Iris Xe integrated GPU. Through extensive benchmarking across various power modes and model optimizations, we provide insights into the trade-offs be-

tween performance, power consumption, and real-time deployment suitability in resource-constrained environments. Furthermore, the application of these models was extended to UAV-based object detection tasks using the VisDrone dataset, enabling an empirical understanding of training strategies that influence detection accuracy in real-world aerial imagery. Finally, we studied the additional impact instance segmentation has on execution time and presented an enhancement on the DeepSORT tracker, which allows for real-time execution even under complex scenes with re-identification enabled.

Through our study and contributions, we achieved the following:

1. Found that only 18 out of the 45 out-of-the-box model configurations were able to run in real time on the NVIDIA Jetson. After TensorRT optimizations, eight out of nine FP32 model configurations succeeded, and nine out of nine did with FP16. Instance segmentation had a big impact on performance, and two out of three models achieved real-time inference even with FP16 optimization included.
2. Demonstrated the large performance and energy efficiency benefits of TensorRT FP16 optimization. At times, we were able to have an almost 4× gain in efficiency.
3. Demonstrated input resolution scaling effects on detection accuracy and inference speed in UAV-relevant scenarios using the VisDrone dataset. Our findings show smaller models, such as the YOLOv12-S, easily winning over their bigger counterparts when adjusting the tensor input.
4. Further tested edge applicability using the Iris Xe graphics device. Five out of nine object detection models managed to run at below 33.3 ms, meeting real-time requirements, and one out of three instance segmentation models passed. Iris Xe also displayed much higher power consumption compared with the Jetson unit.
5. Modernized and improved DeepSORT's inference pipeline, which provides significant improvement in speed and ease of use by deprecating TensorFlow V1. DeepSORT can now run in real-time, even under complex scenes.

The Jetson Nano SUPER platform demonstrated highly efficient performance, particularly when models were executed under the M3 (MAX_N) power mode with jetson_clocks enabled. Notably, lightweight models such as YOLOv10-N and YOLO11-N consistently achieved sub-25 ms inference times, which satisfy real-time requirements. However, the unexpectedly degraded performance observed in M2 mode for certain models emphasizes the influence of underlying thermal and frequency scaling behaviors. These internal management policies may limit GPU utilization under specific thermal or power ceilings, especially for lighter workloads where burst performance is more critical. In addition, a critical insight from our analysis is the narrow performance differential observed between small and medium model variants in the MAX_N mode. Despite their architectural complexity, medium-sized models did not experience a proportionate increase in inference latency, suggesting that preprocessing overhead and data transfer between CPU and GPU constitute a significant portion of the end-to-end pipeline. This highlights the importance of optimizing the entire dataflow, not just the model architecture, to maximize edge inference efficiency. With the application of TensorRT and FP16 precision, Jetson Nano SUPER models achieved dramatic reductions in both inference time and energy consumption. For instance, YOLOv10-N saw a 43.7% latency improvement and over 50% reduction in energy per frame. These findings are critical for deployment in battery-operated systems like UAVs, where power efficiency and thermal constraints are primary concerns. In this mode, all models tested were able to achieve real-time inference, thereby enabling more accurate detections.

Further, model training experiments using the VisDrone dataset revealed the direct impact of training parameters such as input resolution, weight initialization, and warmup strategies on detection accuracy. The YOLOv12-S model achieved its highest mAP (36.9%) at

a 960×960 input resolution, illustrating the advantage of larger receptive fields in capturing small and distant objects common in aerial footage. However, trade-offs in computational complexity and memory usage must be considered when selecting input resolutions for embedded applications. Our findings also underscore the influence of weight transfer and training strategies. Warm starts using pretrained weights significantly improved model accuracy, particularly in configurations with limited training epochs. Moreover, models trained without weight transfer or with suboptimal optimizer settings suffered degraded performance, reaffirming the importance of carefully selected hyperparameters in training deep learning models for edge deployment. Finally, the performance trade-off between model size and tensor input showed that it is often beneficial to use smaller models with higher tensor input for UAV applications. We also believe that these results provide a general benefit to researchers trying to identify the applicability of modern object detection models on edge devices. Power consumption and inference times can apply to any dataset due to the underlying architecture of these models, as their inference time is not related to the size and number of detections per frame.

In the future, we plan to extend this research to more edge devices, such as the Raspberry Pi, using external AI accelerators, as well as dive deeper into why YOLOv12 was at times considerably slower on the Jetson Nano compared with YOLOv11. The literature suggests minimal differences between the two, but this was tested on the NVIDIA T4 Tensor GPU, which is not applicable to edge computing.

Author Contributions: Conceptualization, D.M., I.D., N.P., V.L. and V.K.; methodology, D.M., I.D. and N.P.; software, D.M.; validation, D.M. and N.P.; formal analysis, N.P.; investigation, D.M. and I.D.; resources, D.M. and V.L.; data curation, D.M. and V.L.; writing—original draft preparation, D.M.; writing—review and editing, D.M., N.P. and V.L.; visualization, D.M.; supervision, V.L. and V.K. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: All new data generated are available at https://drive.google.com/drive/folders/1NEoeoOWqN1ftoPJ04cb9ffwDElv_fBIr?usp=sharing. (accessed on 2 August 2025).

Acknowledgments: We would like to acknowledge and thank Vaios Lappas for his guidance and advice. We would also like to thank and show our gratitude for the editorial assistance provided by the Managing Editor. During the preparation of this manuscript, the authors used GenAI tools for the purposes of minor text generation, which aimed to aid in the flow of the paper across the beginning sections. The authors have reviewed and edited the output and take full responsibility for the content of this publication. In addition, the authors have performed syntax and grammar checks throughout the article using publicly available tools. The scientific content was not affected by the use of GenAI tools.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Shakhatreh, H.; Khreishah, A.; Sawalmeh, A.H.; Guizani, M.; Al-Fuqaha, A.; Dou, Z.; Othman, N.S.; Almaita, E.; Khalil, I. Unmanned Aerial Vehicles (UAVs): A Survey on Civil Applications and Key Research Challenges. *IEEE Access* **2018**, *7*, 48572–48634. [CrossRef]
2. Otto, A.; Agatz, N.; Golden, B.; Pesch, E.; Campbell, J. Optimization Approaches for Civil Applications of Unmanned Aerial Vehicles (UAVs) or Aerial Drones: A Survey. *Networks* **2018**, *72*, 411–458. [CrossRef]
3. Lyu, M.; Huang, C.; Huang, H.; Zhao, Y. Unmanned Aerial Vehicles for Search and Rescue: A Survey. *Remote Sens.* **2023**, *15*, 3266. [CrossRef]
4. Boroujeni, S.P.H.; Razi, A.; Khoshdel, S.; Afghah, F.; Coen, J.L.; O'Neill, L.; Fule, P.; Watts, A.; Kokolakis, N.-M.T.; Vamvoudakis, K.G. A comprehensive survey of research towards AI-enabled unmanned aerial systems in pre-, active-, and post-wildfire management. *Inf. Fusion* **2024**, *108*, 102369. [CrossRef]

5. Zualkernan, I.; Abuhani, D.A.; Hussain, M.H.; Khan, J.; Elmohandes, M. Machine Learning for Precision Agriculture Using Imagery from Unmanned Aerial Vehicles (UAVs): A Survey. *Drones* **2023**, *7*, 382. [CrossRef]
6. Zhang, H.; Zhang, Z.; Cai, Y.; Shao, F.; He, X.; Bi, S. Research on Object Detection and Recognition Method for UAV Aerial Images Based on Improved YOLOv5. *Drones* **2023**, *7*, 402. [CrossRef]
7. Patrinopoulou, N.; Lappas, V.; Daramouskas, I.; Meimetis, D.; Kostopoulos, V. *Autonomy in UAV Civilian Applications*; IntechOpen eBooks: London, UK, 2023. [CrossRef]
8. Daramouskas, I.; Lappas, V.; Patrinopoulou, N.; Meimetis, D.; Kostopoulos, V. *Autonomous Systems for Defense Applications*; IntechOpen eBooks: London, UK, 2023. [CrossRef]
9. Patrinopoulou, N.; Daramouskas, I.; Meimetis, D.; Lappas, V.; Kostopoulos, V. A Multi-Agent system using decentralized Decision-Making techniques for area surveillance and intruder monitoring. *Drones* **2022**, *6*, 357. [CrossRef]
10. Meimetis, D.; Daramouskas, I.; Perikos, I.; Hatzilygeroudis, I. Real-time multiple object tracking using deep learning methods. *Neural Comput. Appl.* **2021**, *35*, 89–118. [CrossRef]
11. Wu, W.; Mo, Y.; Liang, Q.; Hu, J.; Xiang, S.; Liu, A.; Duan, P. EUAVDet: An Efficient and Lightweight Object Detector for UAV Aerial Images with an Edge-Based Computing Platform. *Drones* **2024**, *8*, 261. [CrossRef]
12. Hu, M.; Lin, Z.; Tan, H.; Yu, J.; Li, Z.; Wan, X. Efficient-Lightweight YOLO: Improving Small Object Detection in YOLO for Aerial Images. *Sensors* **2023**, *23*, 6423. [CrossRef]
13. Ma, S.; Lu, H.; Liu, J.; Zhu, Y.; Sang, P. LAYN: Lightweight Multi-Scale Attention YOLOv8 Network for Small Object Detection. *IEEE Access* **2024**, *12*, 29294–29307. [CrossRef]
14. Kirac, E.; Özbek, S. Deep Learning Based Object Detection with Unmanned Aerial Vehicle Equipped with Embedded System. *J. Aviat.* **2024**, *8*, 15–25. [CrossRef]
15. Moosmann, J.; Bonazzi, P.; Li, Y.; Bian, S.; Mayer, P.; Benini, L.; Magno, M. *Ultra-Efficient On-Device Object Detection on AI-Integrated Smart Glasses with TinyissimoYOLO*; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2025; pp. 262–280. [CrossRef]
16. Redmon, J.; Divvala, S.; Girshick, R.; Farhadi, A. You Only Look Once: Unified, Real-Time Object Detection. In Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 27–30 June 2016; pp. 779–788. [CrossRef]
17. Wang, C.-Y.; Bochkovskiy, A.; Liao, H.-Y.M. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Vancouver, BC, Canada, 17–24 June 2023.
18. Jocher, G. Ultralytics YOLOv11; AGPL-3.0; 2025. Available online: https://github.com/ultralytics/ultralytics (accessed on 21 May 2025). [CrossRef]
19. Mittal, P. A comprehensive survey of deep learning-based lightweight object detection models for edge devices. *Artif. Intell. Rev.* **2024**, *57*, 242. [CrossRef]
20. Setyanto, A.; Sasongko, T.B.; Fikri, M.A.; Kim, I.K. Near-Edge Computing Aware Object Detection: A Review. *IEEE Access* **2024**, *12*, 2989–3011. [CrossRef]
21. Huang, Z.; Yang, S.; Zhou, M.; Gong, Z.; Abusorrah, A.; Lin, C.; Huang, Z. Making accurate object detection at the edge: Review and new approach. *Artif. Intell. Rev.* **2021**, *55*, 2245–2274. [CrossRef]
22. Vasishta, M.S.S.; Amireddy, A.T.R.; Shrivastava, P.; Srinivasan, R.; Talasila, V.; Shastry, P.N. Small Object Detection for UAVs Using Deep Learning Models on Edge Computing: A Comparative Analysis. In Proceedings of the 2024 5th International Conference on Circuits, Control, Communication and Computing (I4C), Bangalore, India, 4–5 October 2024; pp. 106–112. [CrossRef]
23. Zaidi, S.S.A.; Ansari, M.S.; Aslam, A.; Kanwal, N.; Asghar, M.; Lee, B. A Survey of Modern Deep Learning Based Object Detection Models. *Digit. Signal Process.* **2022**, *126*, 103514. [CrossRef]
24. Raza, S.M.; Abidi, S.M.H.; Masuduzzaman, M.; Shin, S.Y. Survey on the Application with Lightweight Deep Learning Models for Edge Devices. *TechRxiv* **2025**. [CrossRef]
25. Zhu, P.; Wen, L.; Du, D.; Bian, X.; Fan, H.; Hu, Q.; Ling, H. Detection and Tracking Meet Drones Challenge. *IEEE Trans. Pattern Anal. Mach. Intell.* **2021**, *44*, 7380–7399. [CrossRef]
26. Lin, T.-Y.; Maire, M.; Belongie, S.J.; Bourdev, L.D.; Girshick, R.B.; Hays, J.; Perona, P.; Ramanan, D.; Dollár, P.; Zitnick, C.L. Microsoft COCO: Common Objects in Context. *arXiv* **2014**, arXiv:1405.0312.
27. Lv, W.; Xu, S.; Zhao, Y.; Wang, G.; Wei, J.; Cui, C.; Du, Y.; Dang, Q.; Liu, Y. DETRs Beat YOLOs on Real-time Object Detection. *arXiv* **2023**, arXiv:2304.08069.
28. Howard, A.G.; Zhu, M.; Chen, B.; Kalenichenko, D.; Wang, W.; Weyand, T.; Andreetto, M.; Adam, H. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv* **2017**, arXiv:1704.04861. [CrossRef]
29. Zhang, X.; Zhou, X.; Lin, M.; Sun, J. ShuffleNet: An Extremely Efficient Convolutional Neural Network for Mobile Devices. In Proceedings of the 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 18–23 June 2018; IEEE: Piscataway, NJ, USA, 2018; pp. 6848–6856. [CrossRef]

30. Meng, J.; Jiang, P.; Wang, J.; Wang, K. A MobileNet-SSD Model with FPN for Waste Detection. *J. Electr. Eng. Technol.* **2021**, *17*, 1425–1431. [CrossRef]
31. Kumar, S.; Kumar, R.; Saad. Real-Time Detection of Road-Based Objects Using SSD MobileNet-v2 FPNlite with a New Benchmark Dataset. In Proceedings of the 2023 4th International Conference on Computing, Mathematics and Engineering Technologies (iCoMET), Sukkur, Pakistan, 17–18 March 2023; IEEE: Piscataway, NJ, USA, 2023; pp. 1–5. [CrossRef]
32. RangiLyu. NanoDet-Plus: Super Fast and High Accuracy Lightweight Anchor-Free Object Detection Model. 2021. Available online: https://github.com/RangiLyu/nanodet (accessed on 21 May 2025).
33. Wang, C.-Y.; Yeh, I.-H.; Liao, H.-Y.M. YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information. *arXiv* **2024**, arXiv:2402.13616. [CrossRef]
34. Wang, A.; Chen, H.; Liu, L.; Chen, K.; Lin, Z.; Han, J.; Ding, G. YOLOV10: Real-Time End-to-End Object Detection. *arXiv* **2024**. [CrossRef]
35. Wang, C.; He, W.; Nie, Y.; Guo, J.; Liu, C.; Han, K.; Wang, Y. Gold-YOLO: Efficient object detector via Gather-and-Distribute Mechanism. *arXiv* **2023**. [CrossRef]
36. Tian, Y.; Ye, Q.; Doermann, D. YOLOv12: Attention-Centric Real-Time Object Detectors. *arXiv* **2025**, arXiv:2502.12524.
37. Sunsmarterjie. GitHub—Sunsmarterjie/yolov12: YOLOv12: Attention-Centric Real-Time Object Detectors. GitHub. Available online: https://github.com/sunsmarterjie/yolov12 (accessed on 2 August 2025).
38. Lv, W.; Zhao, Y.; Chang, Q.; Huang, K.; Wang, G.; Liu, Y. RT-DETRv2: Improved Baseline with Bag-of-Freebies for Real-Time Detection Transformer. *arXiv* **2024**, arXiv:2407.17140.
39. Wojke, N.; Bewley, A.; Paulus, D. Simple Online and Realtime Tracking with a Deep Association Metric. In Proceedings of the 2017 IEEE International Conference on Image Processing (ICIP), Beijing, China, 17–20 September 2017; pp. 3645–3649. [CrossRef]
40. Xiao, B.; Xiao, H.; Wang, J.; Chen, Y. Vision-Based Method for Tracking Workers by Integrating Deep Learning Instance Segmentation in Off-Site Construction. *Autom. Constr.* **2022**, *136*, 104148. [CrossRef]
41. Kang, J.; Lee, D.; Park, J.-H.; An, T.-H.; Choi, D.; Min, K.-W.; Lee, S. Insta-Track: Instance Segmentation Assisted Tracking by Detection. In Proceedings of the 2024 15th International Conference on Information and Communication Technology Convergence (ICTC), Jeju, Republic of Korea, 16–18 October 2024; pp. 340–345. [CrossRef]
42. Ravi, N.; Gabeur, V.; Hu, Y.-T.; Hu, R.; Ryali, C.; Ma, T.; Khedr, H.; Rädle, R.; Rolland, C.; Gustafson, L.; et al. SAM 2: Segment Anything in Images and Videos. *arXiv* **2024**, arXiv:2408.00714.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.